HW4 readme

Vinisha Bala Dhayanidhi

Q1: create_2gram_hmm.sh / create_2gram_hmm.py
This program counts the bigram transitions and emissions from the training data (BOS at the start of each line, </s>/EOS at the end) and writes the relative-frequency probabilities with log10 values. These values are sorted alphabetically with no smoothing.
Output: 2g_hmm. I verified the approach on wsj_small.word_pos: the header (44 states, 1858 symbols, 1/556/1979 lines) matches wsj_small_hmm.

Q2: check_hmm.sh / check_hmm.py
Data structure: I used nested dictionaries.

- trans[from_state][to_state] = prob
- emit[state][symbol] = prob
- init[state] = prob
- an insertion-ordered dictionary of all the states and a set of all symbols

This gives O(1) lookup for any probability, and each state's outgoing distribution sits in one inner dictionary, so checking that it sums to 1 is a single pass. It is also the structure a Veterbi decoder needs next. The checker compares the header with the real counts (states and symbols as the union across all sections) and warns when the init, transition, or emission sums are not 1. I also tested it against hmm_ex1 and hmm_ex2 and the output matches the example warning files exactly.

Results:

- 2g_hmm.warning: only the expected warnings (EOS has no outgoing transitions and BOS has emission sum 0).
- 3g_hmm_0.1_0.7_0.2.warning: The header claims 9520 emission lines but the file has 9486 and all the states with BOS as 2nd tag have the emission sum as 0.

In this file the _EOS states do have outgoing transitions, which is different from the description in the assignment.

Challenges:

- Rounding: I first wrote probabilities with 6 decimals, which made the per-state sums like 0.999996 produce false warnings in the checker. I fixed this by writing full-precision values.
- Words containing "/": I split each token on its last "/" so that escaped words like 1\/2 will stay intact.